

TaskFlow — Mapa de Arquitectura y Checklist de Implementación

Proyecto: TaskFlow

Stack: Java 21 · Spring Boot · Maven · Spring Data JPA / Hibernate · PostgreSQL · DbGate

Objetivo de aprendizaje: Backend profesional desde cero (sin frontend ni seguridad aún)

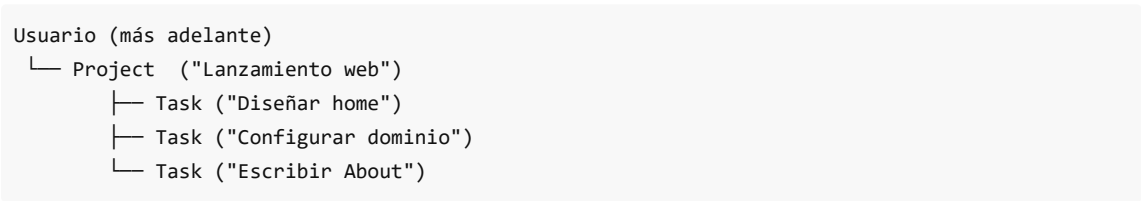
1. Qué es TaskFlow (producto)

TaskFlow es una API backend para organizar **proyectos** y, dentro de ellos, **tareas**.

Ejemplo:

- Proyecto: Lanzamiento web personal
- Tareas: Diseñar home , Configurar dominio , Escribir About

`Project` no es un concepto de Spring. Es un concepto de negocio: un contenedor con nombre que agrupa trabajo.



En el MVP actual no hay login. Eso se agregará en un proyecto posterior.

2. Alcance del MVP

Incluye

- CRUD de proyectos (crear, listar, obtener, actualizar, eliminar)
- Módulo de tareas asociadas a un proyecto
- Validaciones básicas y reglas de negocio simples
- Estructura profesional en capas

No incluye aún

- Usuarios / login / JWT
- Frontend React
- Redis / AWS / microservicios
- Flyway (se implementa en fase de calidad, después del CRUD)

3. Modelo de datos

Tabla `project`

Columna	Significado	Regla
id	Identificador	Lo genera la base de datos
name	Nombre	Obligatorio

description	Descripción	Opcional
created_at	Fecha creación	Se asigna automáticamente

Tabla `task` (siguiente módulo)

Columna	Significado	Regla
id	Identificador	Lo genera la base de datos
title	Título	Obligatorio
done	¿Completada?	Default false
project_id	Proyecto dueño	Obligatorio (FK a project.id)
created_at	Fecha creación	Se asigna automáticamente

Relación

project (1) -----< task (N)
 Un proyecto tiene muchas tareas
 Una tarea pertenece a un solo proyecto

4. Contrato de API

Base URL local: `http://localhost:8080`

Módulo Project

Método	URL	Acción
POST	<code>/api/projects</code>	Crear proyecto
GET	<code>/api/projects</code>	Listar proyectos
GET	<code>/api/projects/{id}</code>	Obtener uno
PUT	<code>/api/projects/{id}</code>	Actualizar
DELETE	<code>/api/projects/{id}</code>	Eliminar

Módulo Task (después)

Método	URL	Acción
POST	<code>/api/projects/{projectId}/tasks</code>	Crear tarea en un proyecto
GET	<code>/api/projects/{projectId}/tasks</code>	Listar tareas del proyecto
PATCH	<code>/api/tasks/{id}</code>	Editar / marcar hecha
DELETE	<code>/api/tasks/{id}</code>	Eliminar tarea

Ejemplo JSON — Crear proyecto

Request POST /api/projects :

```
{
  "name": "Lanzamiento web personal",
  "description": "Tareas para publicar mi sitio"
}
```

Response 201 Created :

```
{
  "id": 1,
  "name": "Lanzamiento web personal",
  "description": "Tareas para publicar mi sitio",
  "createdAt": "2026-08-05T16:00:00"
}
```

5. Estructura profesional del código

```
taskflow/
├─ src/main/java/com/rlalejandro/taskflow/
│   ├── TaskflowApplication.java
│   ├── project/
│   │   ├── Project.java
│   │   ├── ProjectRepository.java
│   │   ├── ProjectService.java
│   │   ├── ProjectController.java
│   │   └─ dto/
│   │       ├── CreateProjectRequest.java
│   │       ├── UpdateProjectRequest.java
│   │       └─ ProjectResponse.java
│   ├── task/ (mismo patrón después)
│   └─ common/ (errores globales, más adelante)
├─ src/main/resources/
│   ├── application.properties
│   ├── application-local.properties (secretos, NO va a Git)
│   └─ db/migration/ (Flyway, más adelante)
```

Responsabilidad por capa

Capa / Archivo	Pregunta que responde
Entity	¿Cómo se guarda este concepto en la BD?
Repository	¿Cómo lo busco / guardo / borro?
Service	¿Se permite hacer esto? ¿Con qué reglas?
Controller	¿Cómo se expone por HTTP?

DTO Request	¿Qué exactamente aceptamos del cliente?
DTO Response	¿Qué exactamente devolvemos al cliente?

6. Orden de funcionamiento de una request

Ejemplo: crear un proyecto.

1. Cliente (Postman) envía POST /api/projects con JSON
2. ProjectController recibe la petición
3. Se valida el formato básico de entrada
4. ProjectService aplica reglas de negocio
5. Si algo falla → error HTTP (400/404/etc.)
6. Si todo ok → ProjectRepository.save(...)
7. Hibernate genera SQL INSERT
8. PostgreSQL guarda la fila en tabla project
9. Vuelve la entidad con id generado
10. Controller arma ProjectResponse (DTO)
11. Cliente recibe JSON 201 Created

Flujo en capas:

HTTP/JSON → Controller → Service (negocio) → Repository → Entity/SQL → PostgreSQL

7. Diagrama lógico del sistema

```

[Postman / futuro React]
  |
  v
[Controllers]      ← puerta HTTP
  |
  v
[Services]         ← reglas de negocio
  |
  v
[Repositories]     ← acceso a datos
  |
  v
[Entities]         ← modelo persistente
  |
  v
[(PostgreSQL)]
  
```

8. Qué significa createdAt (lógica de negocio)

createdAt responde a una pregunta simple:

¿Cuándo nació este registro?

Ejemplos de uso futuro:

- ordenar proyectos del más reciente al más antiguo
- mostrar "creado el 5 ago 2026"
- auditar cuándo se creó algo

Regla de negocio

Cuando se crea un proyecto, el sistema debe registrar automáticamente la fecha/hora de creación.

Quién lo debe poner

No debe depender de que el cliente (Postman) envíe la fecha.

El cliente podría mentir o olvidarla.

Por eso la asigna el backend.

Cómo se implementa en esta fase

Con un callback JPA `@PrePersist` : justo antes del INSERT, si `createdAt` está vacío, se pone la fecha/hora actual.

9. Decisiones técnicas ya tomadas

Tema	Decisión
Build	Maven
Java	21
Persistencia	Spring Data JPA + Hibernate
Base de datos	PostgreSQL
Cliente SQL	DbGate
Secretos	application-local.properties (gitignore)
Perfil actual	local
Esquema ahora	ddl-auto=update
Migraciones luego	Flyway + ddl-auto=none
Estructura	Por feature (project/, task/)
Capas	Entity → Repository → Service → Controller (+ DTOs)
Lombok	Permitido si se entiende qué genera
Seguridad	Fuera de este MVP (proyecto siguiente)

10. Checklist de implementación (archivo por archivo)

Usa esta lista en orden. No saltes capas.

Fase A — Cimientos

- ☒ Crear proyecto Spring Boot (Web + JPA + PostgreSQL)
- ☒ Configurar PostgreSQL y base `taskflow`
- ☒ Separar secretos (`application-local.properties` + `gitignore`)
- ☒ Activar perfil `local`
- ☐ Completar entidad `Project` (Column, PrePersist, getters/setters o Lombok)
- ☐ Arrancar app y verificar tabla `project` en DbGate

Fase B — CRUD Project

- ☐ Crear `ProjectRepository` (interfaz que extiende `JpaRepository`)
- ☐ Crear DTOs:
 - ☐ `CreateProjectRequest`
 - ☐ `UpdateProjectRequest`
 - ☐ `ProjectResponse`
- ☐ Crear `ProjectService` con reglas:
 - ☐ crear
 - ☐ listar
 - ☐ obtener por id (error si no existe)
 - ☐ actualizar
 - ☐ eliminar
- ☐ Crear `ProjectController` con los 5 endpoints
- ☐ Probar todo en Postman

Fase C — Módulo Task

- ☐ Crear entidad `Task` con relación a `Project`
- ☐ Verificar tablas/relación en DbGate
- ☐ `TaskRepository`
- ☐ DTOs de Task
- ☐ `TaskService` (incluye validar que el proyecto exista)
- ☐ `TaskController`
- ☐ Regla de negocio ejemplo: no borrar proyecto con tareas abiertas
- ☐ Probar en Postman

Fase D — Calidad

- ☐ Manejo global de errores
- ☐ Validaciones de entrada (`@Valid` , `@NotBlank` , etc.)
- ☐ Introducir Flyway y apagar `ddl-auto=update`
- ☐ Tests básicos del Service/Controller
- ☐ Inicializar Git correctamente en la carpeta del proyecto
- ☐ Primer commit limpio (sin secretos, sin `.zip`)

11. Orden pedagógico (por qué este orden)

1. **Entity primero** → entiendes el modelo persistente

2. **Repository** → entiendes acceso a datos y beans
3. **Service** → entiendes negocio
4. **DTOs + Controller** → entiendes API HTTP
5. **Task + relaciones** → entiendes modelado real
6. **Flyway / errores / tests** → acercamiento a producción

Si construyes todo junto, cuando falle no sabrás qué capa falló.

12. Glosario rápido

Término	Significado en palabras simples
Entidad	Clase Java mapeada a una tabla
Repository	Contrato para guardar/buscar en BD
Bean	Objeto administrado por Spring
Service	Donde viven las reglas del negocio
Controller	Puerta HTTP de la API
DTO	Objeto de transporte entrada/salida (no es la entidad)
Negocio	Reglas del problema real (no la tecnología)
JPA	Especificación/estándar de persistencia
Hibernate	Motor que implementa JPA y genera SQL
Flyway	Migraciones versionadas del esquema de BD
Perfil local	Configuración solo para tu máquina

13. Estado actual del aprendizaje

Estás en **Fase A — completar entidad Project**.

Siguiente micro-paso:

1. Cerrar `Project` (nullable name + createdAt automático)
2. Verificar tabla en DbGate
3. Pasar a `ProjectRepository`

Documento generado para acompañar el aprendizaje tutorizado de TaskFlow. No sustituye la práctica: implementa el checklist en orden.